

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4

**«Запросы на выборку и модификацию данных. Представления. Работа с
индексами»**

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Уланова Елизавета Эдуардовна

Факультет прикладной информатики

Группа К3241

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2025

Создание таблиц базы данных PostgreSQL. Заполнение таблиц рабочими данными

Цель работы: овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание:

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.
3. Изучить графическое представление запросов и просмотреть историю запросов.
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

Выполнение

№ 1

Этот запрос в Query Tool – он выведет:

- все таблицы;
- все поля;
- типы данных;
- nullable;
- default;
- первичные ключи.

Вывод структуры таблицы

```
SELECT
c.table_name,
c.column_name,
c.data_type,
c.is_nullable,
c.column_default,
CASE
WHEN tc.constraint_type = 'PRIMARY KEY' THEN 'YES'
ELSE 'NO'
END AS is_primary_key
FROM information_schema.columns c
LEFT JOIN information_schema.key_column_usage kcu
ON c.table_name = kcu.table_name
AND c.column_name = kcu.column_name
LEFT JOIN information_schema.table_constraints tc
ON kcu.constraint_name = tc.constraint_name
AND tc.constraint_type = 'PRIMARY KEY'
WHERE c.table_schema = 'public'
ORDER BY c.table_name, c.ordinal_position;
```

	table_name name	column_name name	data_type character varying	is_nullable character varying (3)	column_default character varying	is_primary_key text
1	bus	bus_id	integer	NO	nextval('bus_bus_id_seq'::regclass)	YES
2	bus	license_plate	character varying	YES	[null]	NO
3	bus	manufacture_year	integer	YES	[null]	NO
4	bus	number_of_seats	integer	YES	[null]	NO
5	bus_brand	brand_id	integer	NO	nextval('bus_brand_brand_id_seq'::regclass)	YES
6	bus_brand	name	character varying	NO	[null]	NO
7	bus_brand	country	character varying	YES	[null]	NO
8	bus_brand	manufacturer	character varying	YES	[null]	NO
9	bus_brand	bus_id	integer	NO	[null]	NO
10	bus_type	type_id	integer	NO	nextval('bus_type_type_id_seq'::regclass)	YES
11	bus_type	type_name	character varying	YES	[null]	NO

Вывести список рейсов с маршрутом и автобусом

```
SELECT
    t.trip_id,
    t.trip_number,
    t.trip_status,
    t.departure_date,
    r.route_number,
    r.departure_point,
    r.destination_point,
    r.distance,
    b.license_plate,
    b.number_of_seats
FROM trip t
JOIN route r ON t.route_id = r.route_id
JOIN bus b ON t.bus_id = b.bus_id
ORDER BY t.departure_date;
```

Data Output

Messages

Notifications

Билеты пассажиров

```
SELECT
```

```
    p.passenger_full_name,  
    t.ticket_id,  
    t.seat_number,  
    t.purchase_date
```

```
FROM ticket t
```

```
JOIN passenger_passport pp
```

```
ON t.passport_id = pp.passport_id
```

```
JOIN passenger p
```

```
ON pp.passenger_id = p.passenger_id;
```

	passenger_full_name character varying (100) 🔒	ticket_id integer 🔒	seat_number integer 🔒	purchase_date date 🔒
1	LAB4_Passenger_6	5	6	2026-05-20
2	LAB4_Passenger_11	10	11	2026-05-15
3	LAB4_Passenger_16	15	16	2026-05-10
4	LAB4_Passenger_26	25	26	2026-05-20
5	LAB4_Passenger_31	30	31	2026-05-15
6	LAB4_Passenger_36	35	36	2026-05-10
7	LAB4_Passenger_46	45	46	2026-05-20
8	LAB4_Passenger_51	50	1	2026-05-15
9	LAB4_Passenger_56	55	6	2026-05-10
10	LAB4_Passenger_66	65	16	2026-05-20

Водители рейсов

```
SELECT
```

```
    d.driver_full_name,  
    c.driver_role,  
    tr.trip_number
```

```
FROM crew c
```

```
JOIN driver_passport dp
```

```
ON c.driver_passport_id = dp.driver_passport_id
```

```
JOIN driver d
```

```
ON dp.driver_id = d.driver_id
```

```
JOIN trip tr
```

```
ON c.trip_id = tr.trip_id;
```

VIEW (представления)

Подробная информация о рейсах

```
CREATE VIEW trip_info AS
SELECT
    t.trip_number,
    r.route_number,
    r.departure_point,
    r.destination_point,
    t.departure_date
FROM trip t
JOIN route r
ON t.route_id = r.route_id;
```

	trip_number character varying (20) 🔒	route_number character varying (10) 🔒	departure_point character varying (100) 🔒	destination_point character varying (100) 🔒	departure_date date 🔒
1	LAB4_TRIP_1	LAB4_R2	Город_2	Пункт_2	2026-05-26
2	LAB4_TRIP_2	LAB4_R3	Город_3	Пункт_3	2026-05-27
3	LAB4_TRIP_3	LAB4_R4	Город_4	Пункт_4	2026-05-28
4	LAB4_TRIP_4	LAB4_R5	Город_5	Пункт_5	2026-05-29
5	LAB4_TRIP_5	LAB4_R6	Город_6	Пункт_6	2026-05-30
6	LAB4_TRIP_6	LAB4_R7	Город_7	Пункт_7	2026-05-31
7	LAB4_TRIP_7	LAB4_R8	Город_8	Пункт_8	2026-06-01

Статистика по маршрутам

Query

```
CREATE OR REPLACE VIEW view_route_statistics AS
SELECT
    r.route_id,
    r.route_number,
    r.departure_point,
    r.destination_point,
    r.distance,
    COUNT(DISTINCT tr.trip_id) AS trips_count,
    COUNT(t.ticket_id) AS tickets_count,
    COALESCE(SUM(pr.base_price), 0) AS total_base_price_sum
FROM route r
LEFT JOIN trip tr ON r.route_id = tr.route_id
LEFT JOIN ticket t ON tr.trip_id = t.trip_id
LEFT JOIN price pr ON r.route_id = pr.route_id
GROUP BY
    r.route_id,
    r.route_number,
    r.departure_point,
    r.destination_point,
    r.distance;
```

	route_id integer	route_number character varying (10)	departure_point character varying (100)	destination_point character varying (100)	distance integer	trips_count bigint	tickets_count bigint	total_base_price_sum numeric
1	1	LAB4_R1	Город_1	Пункт_1	105	10	50	13125.00
2	2	LAB4_R2	Город_2	Пункт_2	110	10	48	13200.00
3	3	LAB4_R3	Город_3	Пункт_3	115	10	50	14375.00
4	4	LAB4_R4	Город_4	Пункт_4	120	10	50	15000.00
5	5	LAB4_R5	Город_5	Пункт_5	125	10	50	15625.00
6	6	LAB4_R6	Город_6	Пункт_6	130	10	48	15600.00
7	7	LAB4_R7	Город_7	Пункт_7	135	10	50	16875.00
8	8	LAB4_R8	Город_8	Пункт_8	140	10	50	17500.00

№2

Теперь перейдем к изменению данных в таблицах
Добавляем 100 маршрутов

```

INSERT INTO route (
    route_number,
    departure_point,
    destination_point,
    distance,
    planned_departure_time,
    planned_arrival_time,
    frequency,
    route_type
)
SELECT
    'LAB4_R' || gs,
    'Город_' || gs,
    'Пункт_' || gs,
    100 + gs * 5,
    TIME '08:00' + (gs % 10) * INTERVAL '1 hour',
    TIME '12:00' + (gs % 10) * INTERVAL '1 hour',
    'ежедневно',
    CASE
        WHEN gs % 2 = 0 THEN 'междугородний'
        ELSE 'пригородный'
    END
FROM generate_series(1, 100) AS gs;

```

Добавляем 50 автобусов

```
INSERT INTO bus (  
    license_plate,  
    manufacture_year,  
    number_of_seats  
)  
SELECT  
    'LAB4_BUS_' || gs,  
    2010 + gs % 14,  
    30 + gs % 25  
FROM generate_series(1, 50) AS gs;
```

Добавляем компании-владельцы автобусов

```
INSERT INTO owner_company (  
    company_name,  
    phone,  
    email,  
    address,  
    tin,  
    psrn,  
    bank_details,  
    bus_id  
)  
SELECT  
    'LAB4_Company_' || ROW_NUMBER() OVER (),  
    '+79990000' || ROW_NUMBER() OVER (),  
    'lab4_company_' || ROW_NUMBER() OVER () || '@mail.ru',  
    'LAB4_Address_' || ROW_NUMBER() OVER (),  
    'TIN' || ROW_NUMBER() OVER (),  
    'PSRN' || ROW_NUMBER() OVER (),  
    'LAB4 bank details',  
    b.bus_id  
FROM bus b  
WHERE b.license_plate LIKE 'LAB4_BUS_%';
```

Добавляем цены для маршрутов через подзапрос

```
INSERT INTO price (  
    route_id,  
    assignment_date,  
    base_price  
)  
SELECT  
    r.route_id,  
    CURRENT_DATE,  
    r.distance * 2.5  
FROM route r  
WHERE r.route_number LIKE 'LAB4_R%';
```

Добавляем 1000 пассажиров

```
INSERT INTO passenger (  
    passenger_full_name,  
    date_of_birth,  
    phone,  
    email  
)  
SELECT  
    'LAB4_Passenger_' || gs,  
    DATE '1980-01-01' + (gs % 12000),  
    '+7988000' || gs,  
    'lab4_passenger_' || gs || '@mail.ru'  
FROM generate_series(1, 1000) AS gs;
```

Добавляем паспорта пассажиров

```
INSERT INTO passenger_passport (  
    passenger_id,  
    issue_date,  
    series,  
    number,  
    passport_type  
)  
SELECT  
    p.passenger_id,  
    DATE '2015-01-01' + (p.passenger_id % 3000),  
    'LAB4',  
    p.passenger_id::text,  
    'паспорт РФ'  
FROM passenger p  
WHERE p.passenger_full_name LIKE 'LAB4_Passenger_%';
```


Добавляем 1000 рейсов через подзапросы

```
INSERT INTO trip (  
    trip_number,  
    trip_status,  
    departure_date,  
    actual_departure_time,  
    actual_arrival_time,  
    bus_id,  
    company_id,  
    route_id  
)  
SELECT  
    'LAB4_TRIP_' || gs,  
    CASE  
        WHEN gs % 3 = 0 THEN 'completed'  
        WHEN gs % 3 = 1 THEN 'planned'  
        ELSE 'cancelled'  
    END,  
    CURRENT_DATE + (gs % 30),  
    TIME '08:00' + (gs % 10) * INTERVAL '1 hour',  
    TIME '12:00' + (gs % 10) * INTERVAL '1 hour',  
  
    (  
        SELECT bus_id  
        FROM bus  
        WHERE license_plate LIKE 'LAB4_BUS_%'  
        ORDER BY bus_id  
        OFFSET gs % 50  
        LIMIT 1  
    ),  
  
    (  
        SELECT company_id  
        FROM owner_company  
        WHERE company_name LIKE 'LAB4_Company_%'  
        ORDER BY company_id  
        OFFSET gs % 50  
        LIMIT 1  
    ),  
  
    (  
        SELECT route_id  
        FROM route  
        WHERE route_number LIKE 'LAB4_R%'  
        ORDER BY route_id  
        OFFSET gs % 100  
        LIMIT 1  
    )  
FROM generate_series(1, 1000) AS gs;
```

Добавляем 5000 билетов через подзапросы

```
INSERT INTO ticket (
    status,
    payment_status,
    purchase_type,
    passport_id,
    trip_id,
    seat_number,
    purchase_date,
    return_date
)
SELECT
    CASE
        WHEN gs % 5 = 0 THEN 'returned'
        ELSE 'active'
    END,
    CASE
        WHEN gs % 4 = 0 THEN 'unpaid'
        ELSE 'paid'
    END,
    CASE
        WHEN gs % 2 = 0 THEN 'online'
        ELSE 'office'
    END,

    (
        SELECT passport_id
        FROM passenger_passport
        WHERE series = 'LAB4'
        ORDER BY passport_id
        OFFSET gs % 1000
        LIMIT 1
    ),

    (
        SELECT trip_id
        FROM trip
        WHERE trip_number LIKE 'LAB4_TRIP_%'
        ORDER BY trip_id
        OFFSET gs % 1000
        LIMIT 1
    ),

    1 + gs % 50,
    CURRENT_DATE - (gs % 20),
    NULL
FROM generate_series(1, 5000) AS gs;
```

Обновляем часть данных

UPDATE билетов

```
UPDATE ticket
SET
    status = 'expired',
    return_date = CURRENT_DATE
WHERE ticket_id IN (
    SELECT t.ticket_id
    FROM ticket t
    JOIN trip tr ON t.trip_id = tr.trip_id
    WHERE tr.trip_number LIKE 'LAB4_TRIP_%'
    AND t.status = 'active'
    ORDER BY t.ticket_id
    LIMIT 100
);
```

Удаляем часть данных

DELETE билетов

```
DELETE FROM ticket
WHERE ticket_id IN (
    SELECT t.ticket_id
    FROM ticket t
    JOIN trip tr ON t.trip_id = tr.trip_id
    WHERE tr.trip_number LIKE 'LAB4_TRIP_%'
    AND t.payment_status = 'unpaid'
    ORDER BY t.ticket_id
    LIMIT 50
);
```

Query Query History

```
ANALYZE;
```

QueryQuery HistoryAI Assistant

1EXPLAIN ANALYZE

2SELECT *

3FROM ticket

4WHERE purchase_date = CURRENT_DATE - 5;

Data OutputMessagesNotifications

≡+

▼

▼

SQL

Showing rows: 1 to 5

	QUERY PLAN	
	text	
1	Seq Scan on ticket (cost=0.00..136.62 rows=250 width=43) (actual time=0.021..1.273 rows=250 loop...)	
2	Filter: (purchase_date = (CURRENT_DATE - 5))	
3	Rows Removed by Filter: 4700	
4	Planning Time: 0.617 ms	
5	Execution Time: 1.301 ms	

№3

Теперь создадим индексы

QueryQuery HistoryAI Assistant

CREATE INDEX idx_ticket_purchase_date

ON ticket(purchase_date);

CREATE INDEX idx_trip_departure_date_route_id

ON trip(departure_date, route_id);

Без первого индекса:

Query

Query History

AI Assistant

1

EXPLAIN ANALYZE

2

SELECT *

3

FROM ticket

4

WHERE purchase_date = CURRENT_DATE - 5;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing

	QUERY PLAN	
	text	
1	Seq Scan on ticket (cost=0.00..136.62 rows=250 width=43) (actual time=0.220..1.079 rows=250 loop...	
2	Filter: (purchase_date = (CURRENT_DATE - 5))	
3	Rows Removed by Filter: 4700	
4	Planning Time: 0.395 ms	
5	Execution Time: 1.106 ms	

С индексом

Query

Query History

AI Assistant

1

EXPLAIN ANALYZE

2

SELECT *

3

FROM ticket

4

WHERE purchase_date = CURRENT_DATE - 5;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing rows: 1 to 7

	QUERY PLAN	
	text	
1	Bitmap Heap Scan on ticket (cost=6.22..60.60 rows=250 width=43) (actual time=0.097..0.231 rows=250 loops=1)	
2	Recheck Cond: (purchase_date = (CURRENT_DATE - 5))	
3	Heap Blocks: exact=49	
4	-> Bitmap Index Scan on idx_ticket_purchase_date (cost=0.00..6.16 rows=250 width=0) (actual time=0.074..0.074 rows=250 loo...	
5	Index Cond: (purchase_date = (CURRENT_DATE - 5))	
6	Planning Time: 0.428 ms	
7	Execution Time: 0.270 ms	



idx_ticket_purchase_date



ticket

Теперь рассмотрим второй индекс, без него:

Query Query History AI Assistant

```
1  EXPLAIN ANALYZE
2  SELECT *
3  FROM trip
4  WHERE departure_date = CURRENT_DATE + 10
5         AND route_id = (
6         SELECT route_id
7         FROM route
8         WHERE route_number = 'LAB4_R10'
9         LIMIT 1
10 );
```

Data Output Messages Notifications



Showing row

	QUERY PLAN	
	text	
1	Seq Scan on trip (cost=3.25..35.25 rows=1 width=58) (actual time=0.354..0.355 rows=0 loops=1)	
2	Filter: ((route_id = (InitPlan 1).col1) AND (departure_date = (CURRENT_DATE + 10)))	
3	Rows Removed by Filter: 1000	
4	InitPlan 1	
5	-> Limit (cost=0.00..3.25 rows=1 width=4) (actual time=0.017..0.018 rows=1 loops=1)	
6	-> Seq Scan on route (cost=0.00..3.25 rows=1 width=4) (actual time=0.016..0.017 rows=1 loop...	
7	Filter: ((route_number)::text = 'LAB4_R10'::text)	
8	Rows Removed by Filter: 9	
9	Planning Time: 0.376 ms	
10	Execution Time: 0.376 ms	

Теперь с ним:

QueryQuery HistoryAI Assistant

12345678910

EXPLAIN ANALYZE
SELECT *
FROM trip
WHERE departure_date = CURRENT_DATE + 10
AND route_id = (
SELECT route_id
FROM route
WHERE route_number = 'LAB4_R10'
LIMIT 1
);

Data OutputMessagesNotifications

Showing rows: 1 to 9

Page

QUERY PLAN

text

1Index Scan using idx_trip_departure_date_route_id on trip (cost=3.53..11.55 rows=1 width=58) (actual time=0.071..0.072 rows=0 loo...

2Index Cond: ((departure_date = (CURRENT_DATE + 10)) AND (route_id = (InitPlan 1).col1))

3InitPlan 1

4-> Limit (cost=0.00..3.25 rows=1 width=4) (actual time=0.026..0.027 rows=1 loops=1)

5-> Seq Scan on route (cost=0.00..3.25 rows=1 width=4) (actual time=0.024..0.025 rows=1 loops=1)

6Filter: ((route_number)::text = 'LAB4_R10'::text)

7Rows Removed by Filter: 9

8Planning Time: 0.767 ms

9Execution Time: 0.106 ms

```
graph LR; route[route] --> Limit[Limit]; Limit --> idx["idx_trip_departure_date_route_id"]
```

The diagram illustrates the execution plan for the query. It starts with the 'route' table, which is scanned sequentially. The results are then passed through a 'Limit' operation to restrict the output to one row. Finally, the results are indexed using the 'idx_trip_departure_date_route_id' index.

Вывод

В ходе лабораторной работы были созданы запросы на выборку и модификацию данных в СУБД PostgreSQL, а также разработаны представления для удобного просмотра информации из базы данных. Были изучены планы выполнения запросов с помощью команды EXPLAIN ANALYZE и выполнено сравнение времени работы запросов до и после создания индексов. В результате было подтверждено, что использование индексов позволяет значительно повысить производительность поиска и обработки данных.